Triangle decompositions I: Formalization and reproducibility companion

Juan Pablo Traverso Gianini
Independent researcher, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Companion to Paper I of the series: *Triangle decompositions I: Formalizing the Nash-Williams theorem in Lean* [1], manuscript preprint 1.0.

Paper II of the series: *Triangle decompositions II: Robust absorption for typed triangles* [2].

Preprint 1.0 — 20 October 2026.

Abstract

This companion records the formal scope and reproducibility evidence for Paper I. The selected archive preserves the clean-build evidence for 1,070 project modules and six final axiom queries, and corrects the author headers in its sources. A reversible hash record relates the corrected sources to the compiled bytes; no new Lean build is asserted. We identify the final theorem, fractional results, typed replacement inputs, exact audit target, source alignment and pinned dependencies. The record distinguishes this local compilation from independent reproduction and from the separate Paper II validation. The present editorial update inspects the supplied evidence without running Lean again.

1. Scope and source identity

Paper I formalizes the theorem of Delcourt and Postle [3, Theorem 1.8] that every sufficiently large triangle-divisible graph of minimum degree at least three quarters of its order has a triangle decomposition. The formal development also includes their fractional theorem and stability results. This companion identifies the declarations, the replacement inputs used in the integral proof, and the evidence attached to one fixed source snapshot. It is a technical record, not a substitute for the mathematical proofs in Papers I and II.

The historical clean compilation, before the header correction, finished successfully. Its explicit target was `RequestProject.NW18.AuditPaperIIAssembly`, and the final theorem is `NashWilliams18.PaperI.nash_williams_via_paperII`. All 1,070 project modules in the target's import closure have successful receipts. Their source hashes and project-import object hashes form one consistent chain. The six final axiom queries and proof-dependency audit passed.

These statements concern that target, not every file in the archive or the entire current Paper II tree. Paper I uses the specialized Paper II inputs in this frozen snapshot. The separate incremental Paper II validation recorded in `PAPER_II_LEAN_20261019_v1` passed, including the seventeen conserved families and census exhaustiveness. It is not needed to certify the recorded Paper I snapshot.

The selected source freeze is `PAPER_I_LEAN_20261020_ALIGNED_v3`. Its archive SHA-256 is

`bfa42bd57e6d9c2ac918bbf6fe2c2ba7c01894d1d8387d92e81b090020a6aaff`.

Preprint 1.0 is the publication-series version of this technical document. The earlier private-review documents and the companion stored inside the original archive retain their own version labels. The present document does not overwrite those records or alter the archive. Its release metadata and document hashes are recorded separately.

The preserved build record describes compilation from an empty project output directory, without reusing project objects from earlier builds. The earlier `PAPER_I_LEAN_20261017_v1` remains historical evidence; its final incremental pass rebuilt five modules. Neither record is an independent reproduction by another party. Section 6 describes resumptions within the clean campaign and the use of cached Mathlib build products.

2. Mathematical statement and definitions

The checked main declaration has the quantifier order

$$\begin{gathered}\exists N \in \mathbb{N}\ \forall G : \\ n \geq N, \quad 3 \mid |E(G)|, \quad (\forall v,\ 2 \mid d_G(v)), \quad 3n \leq 4\delta(G) \\ \Longrightarrow \quad G \text{ has a triangle decomposition.}\end{gathered}$$

Here G is a finite simple graph, $n = |V(G)|$, and the order threshold is chosen before the graph. The threshold is exactly $3n/4$, not $(3/4 + \varepsilon)n$. The declaration asserts existence of N; this companion does not supply a numerical value.

`NashWilliams.IsTriangleDivisible` is the conjunction of divisibility of `G.edgeFinset.card` by three and `Even (G.degree v)` for every vertex. `NashWilliams.IsTriangleDecomposition` requires a finite family of three-element cliques and exactly one member containing the endpoints of each edge. `NashWilliams.HasTriangleDecomposition` existentially quantifies that family. These definitions are in `RequestProject/NashWilliamsBasic.lean`.

For comparison with the mathematical statement, the type with namespaces opened is:

```
Exists fun N : Nat =>
  forall (V : Type) [Fintype V] [DecidableEq V]
    (G : SimpleGraph V) [DecidableRel G.Adj],
    N <= Fintype.card V ->
    IsTriangleDivisible G ->
    3 * Fintype.card V <= 4 * G.minDegree ->
    HasTriangleDecomposition G
```

The formal statement additionally carries the finite-type and decidability instances needed to represent a finite graph in Lean. They are representation parameters, not extra graph-theoretic hypotheses. Neither a fractional decomposition nor an absorption certificate is an assumption of the final declaration.

3. Statements available in the selected closure

The paths below are relative to `source/` in the archive. The listed modules all occur in the checked import closure. Inclusion here does not mean that each declaration received a separate final axiom query: Section 5 gives the exact six queried declarations.

Fractional threshold, Delcourt-Postle 1.4. `NashWilliams.Frac.exists_fractional_weights`, in `RequestProject/Fractional/Final.lean`, produces nonnegative real weights on finite vertex sets. For each graph edge, the sum over three-cliques containing its endpoints is

one. Its only graph inequality is `3 * Fintype.card V ≤ 4 * G.minDegree`; neither integral divisibility nor sufficiently large order is assumed.

Fractional stability, Delcourt-Postle 1.6. `NashWilliams.Frac.theorem_1_6_proof`, in `RequestProject/FracStab/Main.lean`, has the order of choices $\exists \sigma_0 > 0$, then $\forall \sigma \in (0, \sigma_0]$, then $\exists \varepsilon > 0$ and n_0. Above that order and degree $(3/4 - \varepsilon)n$, failure of a fractional triangle decomposition implies `IsExtremal σ G`.

Regular fractional stability, Delcourt-Postle 2.6. `NashWilliams18.theorem_2_6`, in `RequestProject/NW18/Theorem26Proof.lean`, also chooses $C \geq 1$ before the graph. Failure of a `C`-regular fractional decomposition implies the same extremality conclusion. The regular decomposition includes exact edge coverage, not just approximate coverage.

Integral theorem, Delcourt-Postle 1.8. `NashWilliams18.PaperI.nash_williams_via_paperII`, in `RequestProject/NW18/PaperIIAssembly.lean`, is the final theorem stated in Section 2 of this companion and in (1.1) of the manuscript. The intermediate declaration `NashWilliams18.PaperI.nash_williams_of_paperII_inputs` still accepts `Branch124Statement` and `Theorem146Statement`; the final declaration supplies both proofs. The distinction matters: an adapter with remaining premises is not the completed result.

Ehard-Glock-Joos matching input. The local declaration used in the proof is `Pseudorandom.theorem_1_2`. Its presence here does not assert that every theorem of that article was audited by the Paper I target. The separate EGJ formalization has its own registration and namespace; the manuscript records it separately. No public package was substituted for the local source during this freeze.

4. The four replacement branches

Delcourt and Postle [3] invoke Theorem 13.16, whose proof they attribute to their manuscript listed as reference 17. A public version of that manuscript had not been located when Paper I was prepared. The claim here is that our selected proof does not depend on Theorem 13.16. We do not infer that the manuscript does not exist, that its theorem is false, or that our theorem is equivalent to an unavailable statement.

Branch of [3]	Replacement in the checked assembly
12.4, non-extremal	The proved protected-round producer, followed by the terminal absorption construction.
12.5 through 14.6	The separate semidense `F0Direct.theorem146_ordered`, supplied through the F0 wrapper.
12.6 and 12.7 through 14.7	The nearly complete typed result `PaddedGeneral.RealInverse.idealized_F12`, supplied through the F1 and F2 wrappers.
12.8, four colours	The new F3 bridge to `PaddedGeneral.RealInverse.nearlyComplete_F123`.

The non-extremal input is `roundUnivSpec124_holds`, passed through `roundSpec124_of_univ` and `branch124_of_round`. It is constructed in the frozen tree rather than postulated. This branch is not an application of nearly complete typed blowups.

The F0 wrapper is `NashWilliams18.PaperII.theorem146_F0`. Its proof uses the separate semidense theorem; the assembly does not pretend that F0 follows from the geometry used for F1 and F2. The implementation counts ordered disagreements: each unordered edited edge contributes two. Thus the manuscript threshold corresponds to twice that number of ordered disagreements, as stated in Paper II, Section 5. The wrappers retain this conversion.

The F1 and F2 wrappers are `NashWilliams18.PaperII.theorem147_F1` and `NashWilliams18.PaperII.theorem147_F2`. In `BlowupPaperInputs.lean`, `idealized_iff_paperII` is proved by `Iff.rfl`: the relevant host predicates are definitionally the same. This is an explicit identification of hypotheses, not a conversion inferred from similar names.

The F3 bridge is `NashWilliams18.PaperII.margin_four_colour_decomposition`, in `PartitionedTriangleAbsorption/PaperII/FourColourApplication.lean`. It takes a proper four-colouring, the stated near-threshold degree condition, even degrees, and the two opposite-pair edge-count balances. Its conclusion supplies an F3 decomposition and the coarsened F0 decomposition of the same edge set. It chooses

$$\varepsilon = \min\{1/16, \eta/3, \delta/4\}.$$

The class-size bounds give a missing-degree bound of $4\varepsilon n$; the floor used for the integer parameter and both inequalities needed to apply it are proved. The final audit requires this bridge and forbids the former `FourColourConsumerStage84.margin_four_colour_decomposition` in the theorem's proof-dependency closure.

In the separate `PAPER_II_LEAN_20261019_v1` freeze, Paper II's final `PaperIApplications` entry queries `PartitionedTriangleAbsorption.FourColourConsumerStage84.margin_four_colour_decomposition`. Paper I excludes that former consumer from its final proof and instead uses `NashWilliams18.PaperII.margin_four_colour_decomposition`, which invokes `nearlyComplete_F123`. The latter has module-level axiom evidence in Paper II, but is not one of its fifteen final entry queries. Paper I's dependency audit and final theorem queries certify the route actually used by Paper I. Different queried targets must not be described as a single synchronized entry point. The aligned Paper I archive certifies the relocated bridge through its own final target. It does not certify a new Paper II `PaperIApplications` audit; that requires the separate Paper II freeze.

5. Axiom and dependency evidence

`AuditPaperIIAssembly.lean` follows constants appearing in the types and proof terms of the final theorem. It checks required replacements and excludes the old global route. This is stronger than finding an import in a source file: an imported declaration may be present without contributing to the final proof.

The six final queries are:

1. `NashWilliams18.PaperII.theorem146_F0`;
2. `NashWilliams18.PaperII.theorem147_F1`;
3. `NashWilliams18.PaperII.theorem147_F2`;
4. `NashWilliams18.PaperII.typed_inputs`;
5. `NashWilliams18.PaperI.nash_williams_of_paperII_inputs`;
6. `NashWilliams18.PaperI.nash_williams_via_paperII`.

Each reported exactly `propext`, `Classical.choice`, and `Quot.sound`. None reported `sorryAx` or an additional mathematical axiom. These are Lean's foundational axioms, so "axiom-free" would be inaccurate. The final audit contains neither errors nor warnings. Module compilation logs do contain warnings; a warning-free project is not claimed.

The dependency audit reports `PASS_PAPERII_PROOF_DEPENDENCIES`. Among its required constants are the F0 theorem, `idealized_F12`, the F3 bridge, `nearlyComplete_F123`, `completeInverseCertLocalBound_general`, and `UniversalProducer.producerTarget_of_threeStepLinks`. Among the forbidden constants are `NashWilliams18.Theorem1316Statement`, the old

nash_williams declaration, the alternative F1 vortex route, and the former F3 consumer. The complete lists are in the frozen audit source.

The v3 archive seal reports PASS_HEADER_ONLY_REVISION. Its 1,070 source headers identify Juan Pablo Traverso Gianini. The historical clean-build evidence is unchanged and applies to the reconstructed pre-correction sources, not to the current source hashes. The internal evidence/verification.json has SHA-256

d83783e3a0dbcc7d3bf1a465460b3996c0889dc6371d362d766e0e48f6583497.

That digest identifies the sealed verification record, not the ZIP or this document. Their digests are recorded separately.

6. Source identity, packages and build scope

The toolchain is leanprover/lean4:v4.35.0-rc3. Mathlib is pinned to c55e6e786f49471c72fbddbec5415808896aec1e; the lock file fixes eight supporting packages. Their revisions are recorded in evidence/environment.json and source/lake-manifest.json. The build used the pinned Mathlib build products. Mathlib itself was not rebuilt.

The selected closure has 1,070 modules: 251 in RequestProject and 819 in PartitionedTriangleAbsorption, including its root module. The archive contains exactly these project sources. It does not certify the full Paper II development.

Before the header-only revision, the alignment starts from the earlier Paper I and Paper II freezes. The 623 previously shared modules take their Paper II source. This resolves the four differences in TypedRefinerF0.PermCount, TypedRefinerF0.Estimates, TypedRefinerF0.RobustHall and BlowupB.PaddedGeneral.RealInverse.RoutedPaper. The 194 Paper I-only modules in that namespace retain their sources. The four-colour application moves, without changing its contents or declaration names, from RequestProject.NW18.FourColourPaperII to PartitionedTriangleAbsorption.PaperII.FourColourApplication; the import in Case128Reduction follows that move. The resulting import closure has one additional module. The alignment manifest records the source origins and edits; it does not claim that every module in the full aligned development was compiled by the Paper I target.

The clean build began with an empty project output directory. It ran in several invocations of tools/build.py paper_I, resumed after resource adjustments. Resumed invocations accepted only objects with successful receipts from this campaign and matching source and object hashes. No project objects from the earlier freezes were reused. The final audit target compiled all six queries and the proof-dependency check.

At sealing, tools/make_freeze.py checked all 1,070 receipts, source and object hashes, and every recorded project-import object hash. The first 23 modules had initially used another Mathlib checkout of the same revision; recompilation against the final checkout produced identical object bytes for all 23. This is a complete local clean compilation, not an independent reproduction by a third party.

7. Reproducing the selected target

The archive preserves the actual campaign scripts. They use Windows-specific memory inspection and fixed workspace paths; they are not a portable command-line wrapper that can be run from an arbitrary extraction directory. They omit Lean executables, compiled project objects and dependency caches.

To repeat the recorded workflow, first inspect tools/align.py and tools/build.py and provide isolated workspace locations for their archive, source and output paths. Supply the two

source archives identified in `evidence/environment.json`, verify their hashes, and install the pinned toolchain. `align.py` reconstructs the aligned source tree and the phase-specific build plan from those archives. Provide Mathlib and the eight supporting packages at their locked revisions, with the corresponding cached build products. Check those revisions before invoking the builder.

The following operations reproduce the historical alignment. To compile the corrected edition, use its supplied `source/` tree with the pinned dependencies and new receipts; do not overwrite it by rerunning the historical alignment script. The campaign operations are:

```
elan toolchain install leanprover/lean4:v4.35.0-rc3
python tools/align.py
python tools/build.py paper_I --workers 4 --threads 3 --memory-mb 5000
```

These commands require the path and dependency preparation just described. The initial invocation must use an empty project output directory. A continuation of the same campaign may use `--resume`; that option is not a substitute for the initial clean start. `tools/make_freeze.py` verifies the source, object and import-hash chain and packages a new record. It must run in a separate output location rather than overwrite the selected archive.

The historical evidence can be inspected without Lean by running `python tools/verify_attribution.py` from the extracted archive root. The script verifies the current hashes and uses `attribution/source_changes.json` to reconstruct each exact pre-correction source from its original header and unchanged body. It compares those reconstructed hashes with all 1,070 historical receipts and checks the 1,671 project-import hash links and the internal manifest. Compiled objects are omitted, so their bytes cannot be rechecked from the archive alone; the producer checked them during the original sealing. No new compilation or object identity for the corrected source hashes is asserted.

8. Reading the archive

`evidence/ALIGNMENT_MANIFEST.json` identifies the source origins and alignment edits. `evidence/build_plan.json` gives the target, its 1,070-module topological order and project imports. `evidence/receipts/` contains source, dependency and object fingerprints; `evidence/module_logs/` contains compiler output for every module. `evidence/final_audit.log` holds the six final queries and dependency success marker. `evidence/verification.json` records the checks made at sealing; `evidence/early_module_recheck.txt` records the 23-module comparison.

`source/` contains the selected closure and pinned build configuration; `tools/` contains the campaign scripts. `SHA256SUMS.txt` binds 3,240 internal files. The archive contains 3,241 files including that manifest, and its external sidecar gives the ZIP digest. Manuscripts and companions remain separate from this Lean archive.

Version v3 adds 1,008 missing author headers, corrects 59 instances of the surname and replaces one attribution to a computational tool with Juan Pablo Traverso Gianini. Two headers were already correct. All 1,070 files now identify that author and Apache-2.0. Every byte after the initial ordinary header comment is preserved, including declarations, proofs and provenance comments. The nine dependency licence texts and all 2,147 pre-existing proof-evidence files are unchanged. `attribution/source_changes.json` binds the old and new hashes; `tools/verify_attribution.py` checks the reversible mapping. The original build README is preserved in `evidence/BUILD_README.md`. The current README and `LICENSING.md` explain this scope; `SHA256SUMS.txt` is regenerated. Cli retains MIT and the other supplied dependency texts retain Apache-2.0.

The English and Spanish documents share the same declarations, mathematical statements, scope and limitations. They use the local Paper I LaTeX template. Document compilation and rendered-page inspection are separate from inspection of the recorded Lean proof evidence.

9. Attribution and preparation for release

Delcourt and Postle are credited for the Nash-Williams theorem and the fractional results formalized here [3]. The replacement mathematics is explained in the two papers. The author's responsibility includes checking that the written statements agree with their formal counterparts. The methods and limits of these replacements must remain visible in the manuscripts, not only here.

Claude, by Anthropic, and ChatGPT/Codex, by OpenAI, were used to explore and check arguments and to prepare the manuscript. Aristotle, by Harmonic, contributed to exploring approaches, searching for counterexamples, developing proofs, and formalizing and reviewing them in Lean. Its role was not limited to translating finished proofs.

Certo [5] and Jacobian [6] were used to explore finite instances, check identities and assess candidate constructions. These computations informed the development of the arguments; finite checks were not treated as proofs of universal statements.

The author remains responsible for the arguments, citations, code and presentation. No AI system is listed as an author.

The selected aligned sources and evidence are in the archive at commit `afa9f4a06b0e`. This reference requires repository access and does not assert public availability. The earlier source archive remains unchanged as a historical record; the current manuscript and companion are separate files.

Paper II preprint 1.0 [2] gives the three-step-link refiner and quantitative assembly in Appendix B, the semidense theorem in Appendix C and the PAR/BAL constructions in Appendix E. Appendix H supplies written proofs of the compatible library and general auxiliary constructions; finite family certificates remain explicit source data. The earlier proof dossiers are retained in [7]. Paper I now uses the aligned sources for its selected typed inputs, checked within its own clean closure. This does not extend its validation to all of Paper II.

The selected formal evidence remains the historical complete local clean compilation. The author-header revision is verified by byte comparison and reversible source reconstruction; it is not a compilation of the corrected source hashes or independent reproduction. Source and dependency licence texts are included. This document and its manuscript remain private until the coordinated release is approved.

Appendix. Main Lean declarations

The fractional theorems, the matching theorem and the typed decomposition inputs are developed separately. The final assembly imports them through the statements used in the case analysis. This makes it possible to compare the hypotheses of each imported result with the properties proved for the residual graph; Sections 4–7 record their source files and dependencies.

The index includes namespaces where a short name would be ambiguous; Sections 4–7 supply the source files, full names of the remaining declarations and dependency records.

Result	Declaration
Main theorem (Delcourt–Postle 1.8)	`nash_williams_via_paperII`
Delcourt–Postle 1.4	`NashWilliams.Frac.exists_fractional_weights`
Delcourt–Postle 1.6	`NashWilliams.Frac.theorem_1_6_proof`
Delcourt–Postle 2.6	`NashWilliams18.theorem_2_6`
Ehard–Glock–Joos 1.2	`Pseudorandom.theorem_1_2`
Proposition 3.1	`NashWilliams18.PaperII.margin_four_colour_decomposition`
Theorem 3.2	`idealized_F12`
Theorem 3.3, as used here	`theorem146_F0`
Proposition 4.1	`regular_seed_nibble`
Proposition 4.2	`roundUnivSpec124_holds`
Corollary 4.3	`branch124_of_round` (conditional on `RoundSpec124`; instantiated below)
Delcourt–Postle 11.4	`Sampling.maxcut_sample`
Delcourt–Postle 14.4	`lemma_14_4_configs_proof`
Lemma A.1	`ParityFill.signed_certificate` (stronger small-degree-deficit version)
Paper II, Lemma SC (ordinary triangles)	`NashWilliams18.absorber_onePart`

The separate EGJ formalization is registered in Palomar as PALOMAR-2026-10-17-000004, version 1 [4]. It covers Theorems 1.1, 1.2, 1.3, 3.1 and 4.1 of the EGJ paper identified in that record; the Nash–Williams proof uses Theorem 1.2. This registration concerns the separate package, not the combined validation of the present development.

For Corollary 4.3, the assembly passes `roundUnivSpec124_holds` through `roundSpec124_of_univ` to `branch124_of_round`; no covering-round assumption is left in the main declaration. This appendix is an index, not the reproducibility record. The toolchain, exact sources, axiom reports and clean-build procedure belong to Sections 4–7.

References

[1] J. P. Traverso Gianini, *Triangle decompositions I: Formalizing the Nash-Williams theorem in Lean*, preprint 1.0, 20 October 2026. Document.

[2] J. P. Traverso Gianini, *Triangle decompositions II: Robust absorption for typed triangles*, preprint 1.0 (2026). Document.

[3] M. Delcourt and L. Postle, *A Proof of Nash-Williams' Conjecture*, arXiv:2606.11178v1 (2026). Bibliographic identification follows Paper I.

[4] Palomar Registry, *PALOMAR-2026-10-17-000004*, version 1 (2026). https://palomar-registry.org/entry.html?id=PALOMAR-2026-10-17-000004&version=1

[5] J. P. Traverso Gianini, *Certo*, software repository. https://github.com/jtraverso/certo-math.

[6] *Jacobian*, software repository. https://github.com/morluto/jacobian.

[7] J. P. Traverso Gianini, *Paper II: written proof dossiers*, private source record, commit 61f752fc87bff6f5654725b23ce10ad4302364ab (19 October 2026). Fixed record.